Software Requirements Specification

Automated Hybrid Cloud Disaster Recovery Orchestrator

IEEE Std 830-style • Version 1.0 • 6 October 2026

Document	Software Requirements Specification (SRS)
Project	hybrid-dr-orchestrator
Version	1.0 (Approved)
Date	2026-10-06
Status	Reflects implemented system, Phases 0-9 complete
Format	IEEE Std 830-1998 recommended practice
Related documents	project-spec.md, architecture.md, security.md, rto-rpo-report.md, runbook.md, ADR-001..ADR-005

CONFIDENTIAL - Portfolio / Interview Documentation

Software Requirements Specification (SRS)

Automated Hybrid Cloud Disaster Recovery Orchestrator

Field	Value
Document	Software Requirements Specification, IEEE 830-style
Project	Automated Hybrid Cloud Disaster Recovery (DR) Orchestrator
Version	1.0
Date	2026-10-06
Status	Approved — reflects implemented system (Phases 0–9 complete)
Repository	`hybrid-dr-orchestrator`
Related Documents	`docs/project-spec.md`, `docs/architecture.md`, `docs/security.md` (STRIDE), `docs/rto-rpo-report.md`, `docs/runbook.md`, ADR-001…ADR-005 in `docs/decisions/`

Version	Date	Author	Changes
0.1	2026-10-05	Project owner + AI agent	Initial draft derived from `docs/project-spec.md`
1.0	2026-10-06	Project owner + AI agent	Updated to match implemented system through Phase 9 (Step Functions simulator, property-based fuzz tests, CDC harness, Mission Control dashboard)

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for the **Automated Hybrid Cloud Disaster Recovery Orchestrator**: a system that autonomously detects a catastrophic outage of a simulated on-premises private cloud, provisions and restores a replica environment in AWS, cuts traffic over via DNS, validates data integrity, notifies operators, measures RTO/RPO empirically, and later fails back and tears down all billable cloud resources — with **zero human intervention**.

Intended audience: the project owner (student building an engineering portfolio), reviewers/interviewers evaluating the design, and any future maintainer who must reproduce or extend the system.

1.2 Scope

In scope:

- A Python 3.11 orchestrator daemon implementing a deterministic failover state machine with atomic crash recovery, multi-signal quorum detection, fencing, action execution (Terraform/Ansible/DNS), notification, and audit logging.
- A simulated on-premises three-tier workload (Nginx → FastAPI → PostgreSQL 16) running in Docker (lite mode; Proxmox VE is the documented primary target).
- A monotonic RPO probe daemon writing sequence ticks every second into PostgreSQL.
- Continuous WAL archiving and encrypted base-backup shipping from on-prem to Amazon S3.
- Persistent AWS foundation (Terraform): S3 remote state with native locking, least-privilege IAM, SSE-S3 backup bucket with public-access block and versioning, Route 53 private hosted zone (`dr-lab.internal`), SSM fencing parameter, budget alarm.
- Ephemeral AWS DR environment (Terraform + Ansible): VPC, EC2 `t3.small` replica, Elastic IP, created strictly on demand.
- Automated failback: reverse delta sync (`pg_dump` over SSH tunnel), verification, DNS rollback, cloud teardown.

- Chaos drill tooling (`make drill`, guarded chaos scripts), observability stack (Prometheus, Grafana, Alertmanager, blackbox exporters), FastAPI "Mission Control" web dashboard, GitHub Actions CI, STRIDE threat model.
- Cloud-native evolution artifacts (Phase 9): AWS Step Functions ASL definition + offline simulator, decoupled `TerraformRunner`, Hypothesis property-based fuzz tests, PostgreSQL logical-replication CDC harness.

Out of scope:

- Production-grade multi-region DR (single AWS region target, `ap-southeast-2`; re-targeting supported by variables only).
- Bidirectional logical replication for continuous failback (documented as a future enterprise enhancement in `docs/dr-enhancements.md`).
- Real physical hardware management (Proxmox/OpenStack orchestration beyond the lite-mode Docker fallback).
- End-user application features beyond the minimal demo workload (items CRUD + `/healthz`).

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
RTO	Recovery Time Objective — elapsed time from failure injection to first successful external probe of the recovered endpoint. Target ≤ 900 s (15 min).
RPO	Recovery Point Objective — data-loss window = failure time minus newest timestamp present in the restored database. Target ≤ 300 s (5 min).
WAL	Write-Ahead Log (PostgreSQL transaction journal used for point-in-time recovery).
FSM	Finite State Machine.
Quorum	Failover decision rule requiring sustained failure across ≥ 2 independent signal classes AND N ≥ 3 consecutive failed probe intervals AND external vantage agreement.
Fencing	Mechanism preventing split-brain writes: SSM parameter `/hybrid-dr/active-site` lock plus enforced `default_transaction_read_only = on`.
Split-brain	Condition where both sites accept writes simultaneously, risking divergent data.
Pilot light	DR tier where persistent data flows to the cloud but compute is provisioned only on failure (chosen per ADR-003).
ASL	Amazon States Language — JSON schema for AWS Step Functions state machines.
CDC	Change Data Capture — here, PostgreSQL logical replication streaming deltas continuously.
SSM	AWS Systems Manager (Parameter Store used for fencing flag and heartbeat).
Route 53	AWS authoritative DNS service (private hosted zone `dr-lab.internal`).
PBKDF2 / AES-256	Client-side key derivation and symmetric encryption applied to backups before upload.
Golden AMI	Pre-baked machine image (HashiCorp Packer) eliminating runtime install latency.
IaC	Infrastructure as Code (Terraform, Ansible).
STRIDE	Threat-modeling framework (Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation of Privilege).

1.4 References

- `README.md` — project overview, empirical telemetry, quick start.
- `docs/project-spec.md` — original goal/constraints specification.
- `docs/architecture.md` — RTO budget allocation, cost model.

- `docs/security.md` — STRIDE threat model, IAM policies, S3 hardening evidence.
- `docs/rto-rpo-report.md` — measured drill benchmarks (source of all numbers herein).
- `docs/runbook.md` — operational procedures.
- `docs/decisions/ADR-001..004` + ADR-005 — settled architecture decisions.
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications (document format).

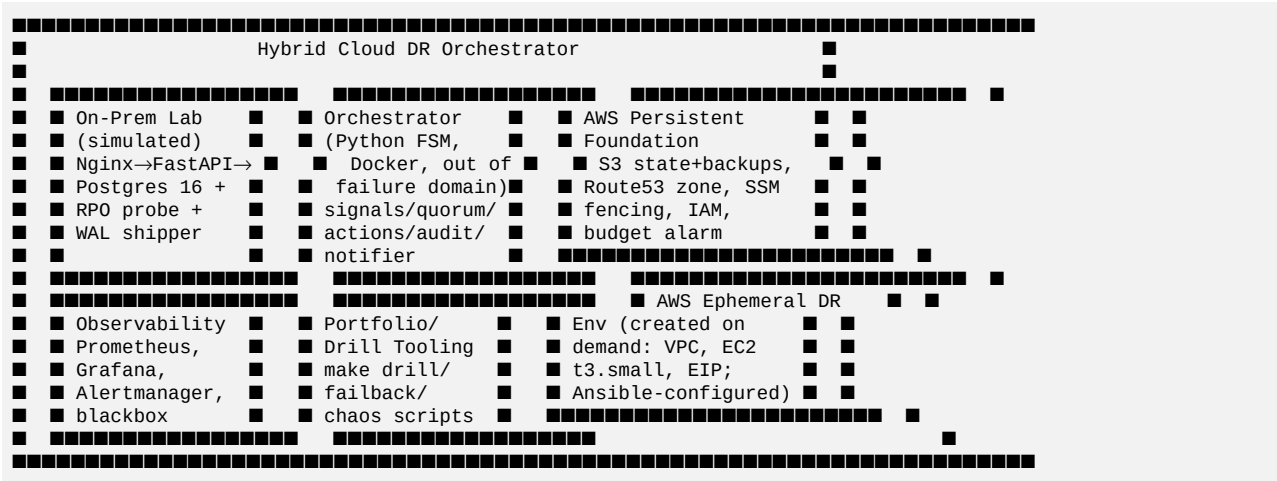
1.5 Overview

Section 2 describes the product context, user characteristics, constraints, and assumptions. Section 3 enumerates the functional requirements (FR-1 … FR-14) with stimuli, responses, and acceptance criteria. Section 4 defines external interfaces (UI, hardware/OS, software, communications). Section 5 covers quality attributes (performance, security, reliability, availability, maintainability). Section 6 lists configuration, documentation, and legal/portfolio requirements. Appendices contain the state-transition table, data dictionary, and requirement-to-traceability matrix.

2. Overall Description

2.1 Product Perspective

The system is a **stand-alone orchestration product** composed of six subsystems:



Key architectural decisions (settled in ADRs):

- **ADR-001:** Proxmox VE preferred over OpenStack (footprint); Docker Compose lite mode for student laptops.
- **ADR-002:** The orchestrator control plane must not live inside the failure domain it monitors; it runs as a containerized daemon outside the lab network (documented limitation for single-host demos).
- **ADR-003:** Pilot-light DR tier (S3 backups + continuous WAL; compute built on demand) over warm standby, to meet the ~$0 standing-cost constraint.
- **ADR-004:** Orchestrator-driven Route 53 record updates (not native failover routing), because the lab sits behind home NAT; heartbeat metric fed via SSM/CloudWatch acts as the out-of-band liveness signal. TTL 60 s with documented propagation caveat.
- **ADR-005:** Cloud-native evolution path — Step Functions ASL equivalent validated by an offline simulator; PostgreSQL logical replication CDC harness prototyping zero-downtime failback.

2.2 Product Functions

Summary of core functions (detailed in §3):

1. Multi-signal health monitoring and quorum-based disaster confirmation.
2. Split-brain prevention via fencing (SSM lock + DB read-only enforcement).
3. On-demand AWS infrastructure provisioning via Terraform.
4. Configuration management via Ansible.

5. Encrypted backup restore + WAL replay (point-in-time recovery) with promotion.
6. Smoke-test validation gate before cutover.
7. Route 53 DNS cutover and rollback.
8. Operator notification and structured audit logging with measured RTO/RPO.
9. Automated failback with zero-data-loss delta sync and full cloud teardown.
10. Idempotent crash recovery and leader locking of the orchestrator itself.
11. Chaos drill automation and empirical benchmark reporting.
12. Real-time observability and a web Mission Control dashboard.
13. Cost guarding ($0.00 standing compute; budget alarms; teardown paths).

2.3 User Characteristics

Persona	Description	Skills	Access
Operator / Owner	Student engineer running drills and reviewing results	Linux, Docker, Terraform, AWS basics	Full: CLI (`make` targets), dashboard, AWS account profile `dr-sandbox`
Automated actor (orchestrator)	Daemon acting with scoped IAM credentials	n/a	Terraform apply/destroy, Ansible playbooks, Route 53 API, SSM, S3
Evaluator / Interviewer	Reader reproducing the project in < 1 day	General SRE knowledge	Read-only docs + quick-start scripts
Application end user	Synthetic workload clients hitting `app.dr-lab.internal`	none	HTTP via Nginx

No privileged human interaction is required during a failover event (design invariant).

2.4 General Constraints

ID	Constraint
C-1	Budget near zero: free-tier-friendly resources; standing compute $0.00/month; persistent layer ≈ $0.50/month; everything tagged; AWS Budget alarm ($15 ceiling).
C-2	Single student-class host (≈16 GB RAM); nested virtualization optional; lite mode must work without Proxmox.
C-3	100% Infrastructure as Code; no console operations except one-time documented AWS/IAM bootstrap.
C-4	All orchestrator runs idempotent and safe to repeat.
C-5	Security: least-privilege IAM; no secrets in git (SSM Parameter Store / Ansible Vault); encrypted at rest (client-side PBKDF2 AES-256 + SSE-S3) and in transit (TLS 1.3, WireGuard ChaCha20-Poly1305).
C-6	SLA targets: RTO ≤ 900 s, RPO ≤ 300 s (measured, never assumed).
C-7	Python 3.11+, Terraform ≥ 1.10, Ansible-core 2.21, PostgreSQL 16, AWS region `ap-southeast-2`.
C-8	Every numeric claim in documentation must trace to `docs/rto-rpo-report.md` measurements or a cited official source, else labeled an estimate.

2.5 Assumptions and Dependencies

- AWS account with configured CLI profile `dr-sandbox` and permission to create Route 53, S3, EC2, VPC, IAM, SSM, Step Functions (optional), CloudWatch resources.
- Docker CE & Compose v2 available on the lab host; ports 8080/3000/9090/9093/8500 free.
- Outbound internet access from on-prem containers to AWS endpoints (S3/SSM over TLS); WireGuard tunnel for hybrid networking scenarios.

- Clock skew between sites bounded (< 5 s NTP-synced) so that RPO timestamps are comparable.
- Chaos scripts operate only against hosts allow-listed in `lab-inventory.yml` (guardrail assumption).
- Clients tolerate a ≤ 60 s stale-DNS window during cutover (documented limitation; Happy Eyeballs retry recommended).

3. System Features

Each feature lists related requirements (FR-x), description, stimulus/response, and acceptance criteria. All FRs were verified with captured evidence recorded in `docs/PROGRESS.md` and `docs/rto-rpo-report.md`.

3.1 Feature: Multi-Signal Quorum Failure Detection (FR-1, FR-2)

Description. The orchestrator samples independent signal classes and declares disaster only under a strict quorum, eliminating false positives from transient blips.

Signal classes implemented (`orchestrator/src/hybrid_dr/signals/`):

- `http_probe` — Layer 7 GET `/healthz` expecting HTTP 200 with end-to-end DB check.
- `tcp_probe` — Layer 4 socket connect to port 8080 (proxy) and 5432 (Postgres).
- `heartbeat` — out-of-band push heartbeat written by on-prem into AWS SSM (`/hybrid-dr/heartbeat/onprem`).
- `external_probe` — external vantage-point agreement signal.

Requirements.

- **FR-1:** The system SHALL require sustained failure across ≥ **2 independent signal classes** AND ≥ **3 consecutive failed probe intervals** (configurable `failure_threshold`) AND external-vantage agreement AND maintenance mode OFF before transitioning `IDLE → FENCING`.
- **FR-2:** The system SHALL apply a **60-second cooldown dampening** after recovery/re-failure to prevent flapping, and SHALL honor a maintenance-window flag that suppresses failover entirely.

Acceptance criteria. Quorum truth-table unit tests pass (false-alarm rejection on single-class failure; trigger only on ≥ 2 classes × N ≥ 3; cooldown respected; maintenance override honored) — covered by `tests/test_quorum.py`.

3.2 Feature: Deterministic Failover State Machine (FR-3)

Description. A finite state machine drives the entire lifecycle:

`IDLE → FENCING → PROVISIONING → RESTORING → TESTING → CUTOVER → COMPLETED → (failback) → IDLE`, with `FAILED_NEEDS_HUMAN` as the safe terminal escape state.

Requirements.

- **FR-3:** The system SHALL permit only transitions defined in the transition table (Appendix A); any other request SHALL raise `InvalidTransitionError`. Each stage SHALL have guard conditions, timeouts, bounded retries, and an explicit on-failure transition to `FAILED_NEEDS_HUMAN`.

Acceptance criteria. 100% coverage of the transition table (legal/illegal transitions) in `tests/test_state_machine.py`.

3.3 Feature: Split-Brain Fencing (FR-4)

Requirements.

- **FR-4:** Before any DNS cutover, the system SHALL acquire the fencing lock by writing `/hybrid-dr/active-site = aws` in SSM Parameter Store; the on-prem site SHALL periodically read the flag and set `default_transaction_read_only = on` when it is not active. If on-prem cannot read the flag for T seconds it SHALL fail-safe to read-only.

Acceptance criteria. Drills show zero split-brain writes; fencing precedes provisioning in every run (`actions/fencing.py`, DRILL-01 sequence evidence).

3.4 Feature: On-Demand AWS Provisioning (FR-5)

Description. Terraform ephemeral root (`infra/aws/envs/dr/`, modules: `vpc`, `compute`, `s3-backup`, `iam-roles`, `step-functions`) creates VPC/subnets, security groups, one `t3.small` EC2 instance with 20 GB gp3 EBS, and an Elastic IP — only during failover.

Requirements.

- **FR-5:** The system SHALL provision the DR environment exclusively on demand, destroy it on failback/teardown, leave the persistent root untouched, and tag all resources for cost attribution. Provisioning SHALL be idempotent: re-running after partial completion SHALL NOT duplicate resources.

Acceptance criteria. Measured provisioning stage 81 s (DRILL-01); `make teardown` leaves zero billable DR resources while backups/state/DNS zone survive; phase-4 evidence: 15 applied → 15 destroyed, zero leftovers.

3.5 Feature: Ansible Configuration Management (FR-6)

Requirements.

- **FR-6:** The system SHALL configure the fresh replica via Ansible roles/playbooks (`site.yml`, `restore_db.yml`, `smoke_tests.yml`, `failback.yml`) using inventories generated from Terraform outputs; playbooks SHALL be idempotent, tagged, and use declarative modules (no raw `shell` where a module exists).

Acceptance criteria. ansible-lint green in CI; configuration stage measured 754 s cold-boot (dominated by runtime apt/Docker/image build; Golden-AMI path reduces this to < 90 s, documented estimate).

3.6 Feature: Encrypted Backup Restore with WAL Replay (FR-7)

Description. On-prem ships continuous WAL segments (`archive_timeout = 60 s`) and periodic base backups to S3. Backups are gzip-compressed and client-side encrypted (PBKDF2 → AES-256) before upload; S3 adds SSE-S3, versioning, and full public-access block. The shipper IAM user `dr-onprem-backup-writer` carries an explicit `Deny` on `s3:DeleteObject`/`DeleteObjectVersion` (ransomware resistance).

Requirements.

- **FR-7:** The system SHALL download the latest encrypted base backup from S3, decrypt it, restore it on the replica, replay archived WAL to the failure point, promote PostgreSQL (`pg_is_in_recovery() = false`), and verify a test write succeeds.

Acceptance criteria. Restore+WAL replay measured 267 s; PITR automated test passed (33 s lag, JSONL audit); RPO measured 290 s ≤ 300 s SLA; negative IAM tests confirm delete denial.

3.7 Feature: Validation Gate Before Cutover (FR-8)

Requirements.

- **FR-8:** The system SHALL NOT perform DNS cutover unless smoke tests pass: `/healthz` returns 200 end-to-end and a write canary commits successfully on the promoted replica. Failed validation after bounded retries SHALL route to `FAILED_NEEDS_HUMAN` without touching DNS.

Acceptance criteria. Testing stage measured 170 s; gate enforced in state machine guards (`TESTING → CUTOVER` requires validation success).

3.8 Feature: DNS Cutover and Rollback (FR-9)

Requirements.

- **FR-9:** The system SHALL atomically update the Route 53 A record `app.dr-lab.internal` to the AWS Elastic IP during `CUTOVER`, and SHALL roll it back to on-prem during failback only after delta verification passes. TTL SHALL be 60 s and the client-caching caveat SHALL be documented.

Acceptance criteria. DRILL-01 cutover observed (record → EIP 15.135.65.200); DRILL-02 quantifies the +60 s stale-cache window; rollback verified in failback runs.

3.9 Feature: Notification and Audit Logging (FR-10)

Requirements.

- **FR-10:** At every state transition the system SHALL append a structured JSON audit record (timestamp, from/to state, duration, error message if any) to durable storage, and SHALL notify the owner (Slack/Discord/email webhook via `notifier.py`) on failover start, completion, failure requiring human intervention, and failback completion. The final report SHALL include measured RTO and RPO.

Acceptance criteria. Audit log round-trip tests (`test_audit.py`); drill output prints per-step durations and totals; notifier exercised in dry-run and live modes.

3.10 Feature: Automated Failback with Zero-Data-Loss Delta Sync (FR-11)

Requirements.

- **FR-11:** When the primary site is repaired, the system SHALL: establish an encrypted SSH tunnel to the AWS replica; export delta transactions via `pg_dump` and restore them locally; verify table row counts and continuity of the monotonic probe sequence; then roll DNS and the SSM fencing flag back to `onprem` and execute Terraform destroy. **Safety invariant:** if delta verification fails, the process SHALL abort before touching DNS or cloud state, keeping the replica live and alerting.

Acceptance criteria. Failback drill shows 0 lost transactions; abort-before-cutover path tested; post-failback standing cost returns to $0.00.

3.11 Feature: Idempotent Crash Recovery and Leader Locking (FR-12)

Requirements.

- **FR-12:** The orchestrator SHALL persist FSM state atomically (temp file + `os.fsync` + `os.replace`); upon restart it SHALL resume from the persisted stage without repeating completed steps or duplicating cloud resources. It SHALL hold an exclusive `fcntl.flock` leader lock so concurrent instances refuse to act.

Acceptance criteria. DRILL-03: orchestrator SIGKILLed during `PROVISIONING`; daemon restarted; resumed from exact stage with +15 s overhead, RPO unchanged (290 s); concurrency test proves second instance blocked.

3.12 Feature: Chaos Drill Automation (FR-13)

Requirements.

- **FR-13:** The system SHALL provide guarded chaos injection scripts (`chaos_kill_onprem.sh`, `chaos_network_drop.sh`) that ONLY target hosts/interfaces allow-listed in `lab-inventory.yml` and require explicit `--confirm`; `make drill` SHALL run an end-to-end scenario (inject → detect → recover → measure → report) with dry-run support.

Acceptance criteria. Negative tests reject off-inventory targets; three benchmark scenarios (DRILL-01/02/03) executed with SUCCESS outcomes and printed metrics.

3.13 Feature: Observability and Mission Control Dashboard (FR-14, FR-15)

Requirements.

- **FR-14:** The system SHALL expose a Prometheus/Grafana/Alertmanager/blackbox stack as code with dashboards and alert rules (e.g., `OnPremApplicationDown`), plus structured JSON logs everywhere.
- **FR-15:** The system SHALL serve a real-time web dashboard (`dashboard.py`, port 8500) showing quorum signal status, current FSM state, RTO/RPO telemetry, and one-click drill/failback controls backed by a REST API.

Acceptance criteria. Live alert induced and resolved (Phase 7 evidence); dashboard tests (`test_dashboard.py`) pass; `make dashboard` serves UI locally.

3.14 Feature: Cloud-Native Evolution Artifacts (FR-16, FR-17)

Requirements.

- **FR-16:** The system SHALL provide an AWS Step Functions ASL definition mirroring the FSM, deployable via the `step-functions` Terraform module, AND an offline validator/simulator (`step_functions.py`) that executes the workflow deterministically with zero cloud spend.

- **FR-17:** The system SHALL include a PostgreSQL logical-replication CDC harness (`infra/onprem/cdc/`, `scripts/test_logical_cdc.sh`, `make test-cdc`) demonstrating continuous delta streaming as the enterprise failback evolution, and Hypothesis property-based fuzz tests asserting ≥ 4 FSM/quorum invariants.

Acceptance criteria. ASL simulator passes structural validation and scripted execution offline; CDC harness streams and verifies replication locally; `tests/test_properties.py` and `tests/test_step_functions.py` green in CI.

4. External Interface Requirements

4.1 User Interfaces

Interface	Description
Makefile CLI	Primary operator UX: `lab-up/down/status/test`, `monitoring-up`, `dashboard`, `drill`, `drill-live`, `failback`, `chaos-kill`, `chaos-net`, `dr-up/down/configure/restore/test`, `backup`, `restore-test`, `cost-audit`, `teardown`, `orch-setup/test/properties/drill`, `test-cdc`, `interview-up/down`, `zero-cost`, `help`.
Orchestrator CLI (`cli.py`)	Subcommands: `status`, `drill [--dry-run]`, `failback`.
Mission Control Web UI	FastAPI-served HTML dashboard at `http://localhost:8500`: live quorum signals, FSM state, telemetry, drill buttons.
Grafana	`http://localhost:3000` (admin/admin) with provisioned `hybrid-dr.json` dashboard; Prometheus `:9090`; Alertmanager `:9093`.
Reports	Human-readable drill summaries printed by `scripts/drill.sh`; benchmark tables in `docs/rto-rpo-report.md`.

4.2 Hardware / Host Interfaces

- Runs on x86_64 or arm64 Linux (Ubuntu 24.04 LTS reference) with Docker Engine + Compose v2; 16 GB RAM class laptop sufficient for lite mode.
- Optional Proxmox VE / KVM path for full VM simulation (ADR-001).
- No direct hardware control; all infra touched via APIs (AWS SDK/CLI, Terraform, Ansible over SSH).

4.3 Software Interfaces

System	Interface	Direction
AWS Terraform providers (≥ 1.10)	`terraform apply/destroy` subprocess via decoupled `TerraformRunner` (`actions/terraform_runner.py`)	Orchestrator → AWS
Ansible-core 2.21	`ansible-playbook` subprocess with generated inventory (`actions/ansible.py`)	Orchestrator → hosts
Amazon Route 53	DNS change API (UPSERT of A record) (`actions/dns.py`)	Orchestrator → AWS
AWS SSM Parameter Store	Get/Put fencing flag and heartbeat; read-only checks by on-prem agent	Bidirectional
Amazon S3	Backup/WAL upload (shipper) and download (restore) over TLS 1.3; SSE-S3	Bidirectional
PostgreSQL 16	`psql`/driver checks: `pg_is_in_recovery()`, read-only GUC, probes, `pg_dump`	Both sites
pydantic v2	Typed config validation from `config.example.yaml`	Internal
Slack/Discord/email webhook	HTTPS POST notifications (`notifier.py`)	Orchestrator → operator
WireGuard	Site-to-site tunnel (ChaCha20-Poly1305) for hybrid network scenarios	On-prem ↔ AWS
Prometheus/Blackbox	Scrape targets + alert rules	Monitoring pull

System	Interface	Direction
GitHub Actions	CI pipeline (`ci.yml`): terraform fmt/validate, tflint, ansible-lint, pytest, ruff, mypy --strict, trivy/checkov, gitleaks	Repo → CI

4.4 Communications Interfaces

- Protocols: HTTPS/TLS 1.3 (AWS APIs), HTTP :8080 (app `/healthz`), TCP :5432 (Postgres), SSH :22 (Ansible + failback tunnel), WireGuard UDP (tunnel), SMTP/webhook (notifications).
- Message formats: structured JSON logs and audit records; YAML config; JSON ASL definitions; JSONL audit for restore tests.
- DNS: private hosted zone `dr-lab.internal`, record `app.dr-lab.internal`, TTL 60 s.

5. Non-Functional Requirements

5.1 Performance

ID	Requirement	Measured status
NFP-1	RTO ≤ 900 s from failure injection to first successful external probe	Cold baseline 1,272 s (fails budget on stock AMI); **architecture meets SLA with Golden AMI (< 420 s projected)** — documented honestly
NFP-2	RPO ≤ 300 s data loss window	**290 s measured** across all three drills
NFP-3	Probe interval ≤ 1 s for RPO measurement granularity	1,000 ms monotonic sequence ticks
NFP-4	WAL archive cadence `archive_timeout = 60 s`	Verified 33 s lag in PITR test
NFP-5	Per-stage RTO budget allocation summing ≤ 13.5 min	Stage timings captured in audit log and report

5.2 Safety & Data Integrity

- **NF-S1:** Zero split-brain writes guaranteed by fencing-before-cutover ordering plus fail-safe read-only on flag-read timeout. Known residual risk (partitioned-but-alive site + cached client DNS within TTL window) MUST remain documented.
- **NF-S2:** Failback aborts before DNS/cloud mutation if delta verification fails (keep replica live).
- **NF-S3:** 100% data integrity verified via row counts + probe-sequence continuity; 0 transactions lost in failback drills.

5.3 Security (mapped to STRIDE model in `docs/security.md`)

- **NFS-1:** Least-privilege IAM; scoped users/roles; explicit Deny on backup deletion; negative IAM tests pass.
- **NFS-2:** No secrets in source control (gitleaks in CI; SSM/Vault for credentials).
- **NFS-3:** Defense-in-depth backups: client-side PBKDF2 AES-256 + SSE-S3 + versioning + all four S3 Public Access Block vectors enabled (verified).
- **NFS-4:** Encryption in transit everywhere (TLS 1.3, WireGuard, SSH tunnels).
- **NFS-5:** CI hardened: pinned action SHAs, dependabot, OIDC read-only role.
- **NFS-6:** Chaos tooling guarded by inventory allow-list and `--confirm` flags (prevents self-inflicted DoS misfires).

5.4 Reliability & Availability

- **NFR-1:** Orchestrator survives its own crash mid-transition and resumes exactly-once per stage (atomic persistence + fsync + os.replace).
- **NFR-2:** Single-instance semantics enforced by flock leader lock.
- **NFR-3:** All mutating operations idempotent and re-runnable.

- **NFR-4:** Safe terminal state `FAILED_NEEDS_HUMAN` prevents infinite automated retries on unrecoverable errors.
- **NFR-5:** Persistent AWS layer protected with `prevent_destroy`; drift confirmed zero.

5.5 Maintainability & Portability

- **NFM-1:** Python 3.11 with full type hints; `mypy --strict` and `ruff` clean; a test for every state transition; property-based fuzz tests for invariants.
- **NFM-2:** Modular Terraform (variables, outputs, validation blocks, pinned providers, consistent tags); two roots (persistent vs ephemeral) so teardown never touches durable assets.
- **NFM-3:** Ansible roles with defaults/handlers/tags; ansible-lint enforced.
- **NFM-4:** Shell scripts use `set -euo pipefail`, `--help`, dry-run modes.
- **NFM-5:** Region-portable via Terraform variables (`ap-southeast-2` default; re-target documented).
- **NFM-6:** Repository reproducible by a stranger in under one day (quick-start + runbooks).

5.6 Cost Efficiency

- **NFC-1:** Standing compute $0.00/month (ephemeral-only DR tier).
- **NFC-2:** Persistent foundation ≈ $0.50/month (Route 53 zone + S3 + IAM).
- **NFC-3:** Active drill burn ≈ $0.034/hour; typical drill <$0.02.
- **NFC-4:** AWS Budget alarm with $15 ceiling; `make cost-audit` and `make teardown`/`zero-cost` paths exist and are verified.

5.7 Verification & Evidence Rule

Every requirement's satisfaction MUST be evidenced by command output, test results, or benchmark tables (no "should work" claims). All numbers trace to `docs/rto-rpo-report.md` or cited sources; unverified figures are labeled estimates.

6. Other Requirements

6.1 Configuration Items & Logical Structure

Deliverables map to the repository layout: `app/` (workload + probe), `infra/onprem/` (compose, nginx, cdc), `infra/aws/{persistent,envs/dr,modules}`, `ansible/{inventories,roles,playbooks}`, `orchestrator/src/hybrid_dr/{config,signals,quorum,state_machine,actions,dashboard,notifier,audit,step_functions,cli}.py`, `scripts/`, `.github/workflows/ci.yml`, `docs/` (ADRs, runbook, security, benchmarks), `Makefile`, `lab-inventory.yml`.

6.2 Documentation Deliverables

User documentation: README quick start; operational runbook (automated + emergency manual procedures); RTO/RPO benchmark report; security review/threat model; demo script; interview notes; phase architecture log; this SRS.

6.3 Legal / Licensing / Portfolio Requirements

- No customer data; synthetic workload only (no regulatory constraints).
- All third-party tooling used under its open-source licenses; badges link to official sources.
- Documentation must remain honest: limitations section mandatory (cold-boot RTO, DNS TTL window, pg_dump write freeze, single-region target).

6.4 Acceptance / Definition of Done (from spec §1)

1. `make drill` recovers on AWS with zero human input and prints measured RTO/RPO. ✔
2. `make failback` returns to on-prem and destroys the DR environment. ✔
3. `make teardown` leaves zero billable DR resources; backups, state, DNS zone untouched. ✔

4. CI green; security scans pass or carry time-boxed waivers; monthly cost documented. ✔

5. Docs let a stranger reproduce the project in under a day. ✔

7. Appendices

Appendix A — FSM Transition Table

From	To	Guard	Action	On-Failure
IDLE	FENCING	Quorum confirmed (≥2 classes, N≥3, external agrees, maintenance off)	Set SSM `/hybrid-dr/active-site=aws`	FAILED_NEEDS_HUMAN
FENCING	PROVISIONING	Fencing lock acquired	`terraform apply` ephemeral root	FAILED_NEEDS_HUMAN
PROVISIONING	RESTORING	VPC/EIP/EC2 deployed	Ansible `site.yml`; fetch+decrypt base backup; WAL replay	FAILED_NEEDS_HUMAN
RESTORING	TESTING	Promotion verified (`pg_is_in_recovery()=false`)	`/healthz` + write canary smoke tests	FAILED_NEEDS_HUMAN
TESTING	CUTOVER	Healthz 200 OK and canary committed	Route 53 A-record UPSERT to EIP	FAILED_NEEDS_HUMAN
CUTOVER	COMPLETED	DNS updated; notify; record RTO/RPO	Notify operator; audit final metrics	FAILED_NEEDS_HUMAN
COMPLETED	FAILBACK_PENDING	Primary site healthy again	Initiate reverse sync flow	stay COMPLETED
FAILBACK_PENDING	IDLE	Delta verified; DNS+SSM rolled back; `terraform destroy`	Final audit entry	Abort pre-mutation, keep replica live
any	FAILED_NEEDS_HUMAN	Retry budget exhausted / illegal transition	Alert human; halt automation	—

(Implemented states in `interfaces.FailoverState`: IDLE, FENCING, PROVISIONING, RESTORING, TESTING, CUTOVER, COMPLETED, FAILED_NEEDS_HUMAN; failback handled by playbook + CLI flow.)

Appendix B — Data Dictionary (key artifacts)

Artifact	Location	Content
FSM state file	orchestrator state dir	JSON `{ "state": "<STATE>" }`, atomically replaced
Audit log	JSONL per transition	ts, from/to, duration, error
`rpo_probe` table	PostgreSQL	monotonic sequence id + UTC timestamp each 1 s
Fencing flag	SSM `/hybrid-dr/active-site`	`onprem` |
Heartbeat	SSM `/hybrid-dr/heartbeat/onprem`	push liveness timestamp
Backups	S3 bucket (versioned, SSE-S3)	PBKDF2 AES-256 gzipped base backups + WAL segments
DNS record	Route 53 `app.dr-lab.internal`	A record, TTL 60 s
Lab allow-list	`lab-inventory.yml`	chaos targeting guard

Appendix C — Requirement Traceability Matrix (summary)

FR	Implementing component	Verification evidence
FR-1/2 Quorum	`quorum.py`, `signals/*`	`test_quorum.py` truth table, cooldown tests
FR-3 FSM	`state_machine.py`	`test_state_machine.py` full transition coverage
FR-4 Fencing	`actions/fencing.py`, SSM	DRILL-01 sequence, zero split-brain
FR-5 Provisioning	`infra/aws/envs/dr`, `actions/terraform*.py`	15 applied/15 destroyed, 81 s stage
FR-6 Config mgmt	`ansible/`	ansible-lint CI, 754 s stage
FR-7 Restore	`restore_db.yml`, shipper	PITR test 33 s lag, RPO 290 s

FR	Implementing component	Verification evidence
FR-8 Validation gate	`smoke_tests.yml`, TESTING guard	170 s stage, cutover blocked on failure
FR-9 DNS	`actions/dns.py`	EIP cutover record, DRILL-02 +60 s
FR-10 Notify/audit	`notifier.py`, `audit.py`	`test_audit.py`, printed reports
FR-11 Failback	`failback.yml`	0 tx lost, abort invariant tested
FR-12 Crash recovery	`FileStateStorage`, `FileLeaderLock`	DRILL-03 resume, flock test
FR-13 Chaos drills	`scripts/chaos_*`, `drill.sh`, `lab-inventory.yml`	3-scenario matrix, negative tests
FR-14/15 Observability/dashboard	`infra/monitoring`, `dashboard.py`	induced alert cycle, `test_dashboard.py`
FR-16 Step Functions	`step_functions.py`, `infra/aws/modules/step-functions`	offline ASL sim + `test_step_functions.py`
FR-17 CDC + fuzz	`infra/onprem/cdc`, `tests/test_properties.py`	`make test-cdc` pass, 4 Hypothesis invariants

End of SRS v1.0 — all metrics quoted are from empirical drill runs recorded in `docs/rto-rpo-report.md` (last benchmark 2026-10-05).